

Exceptional Handling in Python

①

Errors -

III) Types of Error - Syntax Error or compile time error
↳ logical error → o/p is wrong
↳ Runtime Error → prog. terminates unexpectedly.

I) Need of Exception Handling

- The prog terminates whenever an error occurs
- Sudden termination can corrupt the program.
- Exception may cause data loss.

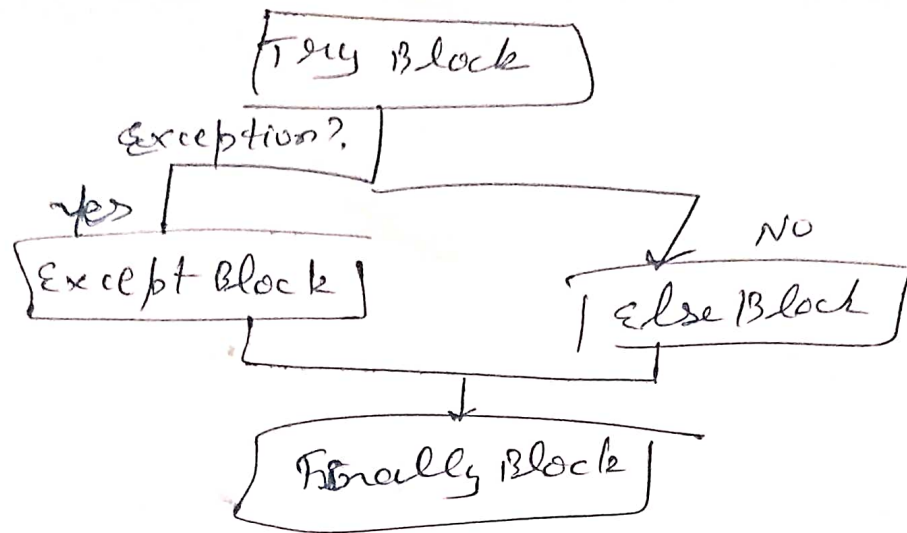
II) What is Exception Handling?

- An exception is a runtime error that terminates the execution of a prog.
- Exception handling generates a message in correspondence to the occurrence of an error & takes the right step to handle it.

→ How Exception Handling works?

```
try: { run the code
except: { error message or piece of code when there is an exception
else: { No Exceptions? Run this code
finally: { Always run the code
```

(2)



Points to Remember

- You can write try block without any except blocks
- You can write several except blocks for a single try block.
- You can write multiple except blocks to handle multiple exceptions.
- You can write exception as an object.
- You can write multiple exception within tuple.

Different types of Exceptions in Python:

- Several Built-in Python exceptions that can be raised when an error occurs during the execution of a program.
- Common types of exceptions in Python: -
 - 1) Syntax Error -
 - 2) Type Error -

NameError -IndexError -KeyError -ValueError -AttributeErrorIO ErrorZero Division ErrorImport ErrorTry --- Except blockSyntaxtry:
code that may cause exception

except:

code to run when exception occurs

⇒ When an exception occurs, it is caught by the except block
 The except block can't be used without the try block

eg try:
 numerator = 10
 denominator = 0
 result = numerator / denominator
 print(result)

except:
 print("Error: Denominator can't be 0")

O/P - Error: Denominator can't be zero

when an exception occurs, the rest of the code inside the try block is skipped. (4)

→ The except block catches the exception & statements inside the except block are executed.

→ If none of the statement in the try block generates an exception, the except block is skipped.

1) try with multiple except block.

Ex
try:
even_numbers = [2, 4, 6, 8]
print(even_numbers[5])

except ZeroDivisionError:
print("Denominator can't be zero")

except IndexError:
print("Index out of Bond")

o/p Index out of Bond

Try with else clause

→ In some situations, we might want to run a certain block of code if the code block inside try runs without any errors.

→ we can use optional else keyword with the try statement

Ex
try:
num = int(input("Enter a no. :"))
assert num % 2 == 0
except:
print("Not an even number!")
else:

(5)

```
reciprocal = 1/num
print(reciprocal)
```

O/P-1 Enter a number: 1
odd no.
Not an even number!

O/P-2 Enter a number: 4
even no.
0.25
→ reciprocal is computed

O/P-3 If we pass 0 (zero) we get
ZeroDivisionError as

Note: Exceptions in the else clause are not handled by the preceding except clause.

try - finally

⇒ finally block is always executed no matter whether there is an exception or not.
It is optional, & for each try block there can be only one finally block.

CS
try:
numerator = 10
denominator = 0
result = numerator/denominator
print(result)

except:
print("Error: Denominator can't be 0")

finally:
print("This is finally block")

error: denominator can't be 0.
This is finally block.